

ArchElab - Lab 1

Corso A, Turno 2 di laboratorio

Cognomi A-D, matricola pari

Docente: Maurizio Lucenteforte

Ricevimento: su appuntamento, inviare email a
maurizio.lucenteforte@unito.it

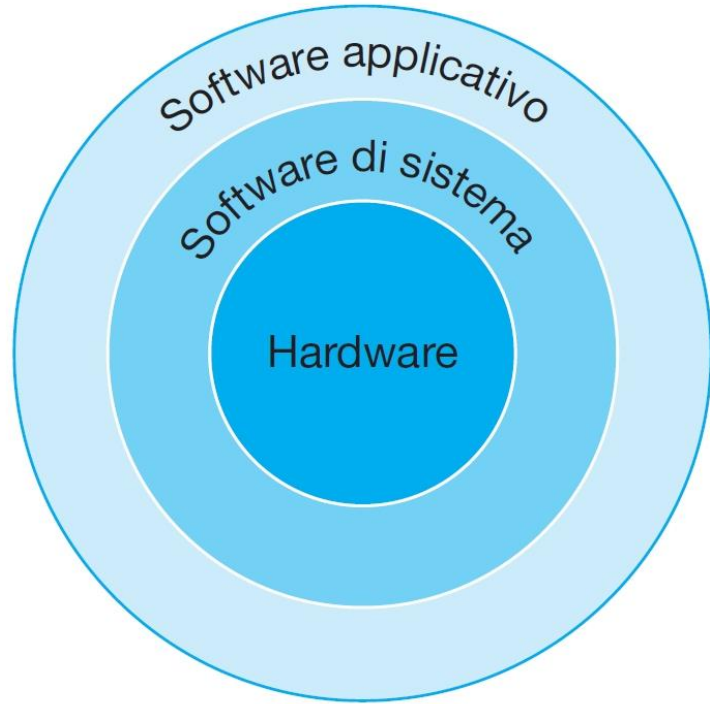
ArchElab - Lab 1

Le istruzioni RISC-V e il simulatore RARS

Obiettivi

- Scrivere i nostri primi programmi in linguaggio RISC-V
- Conoscere il simulatore RISC-V RARS
- Simulare l'esecuzione dei programmi RISC-V usando RARS

Relazione gerarchica tra hardware e software



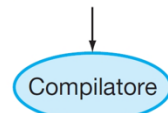
- Relazione gerarchica tra **hardware** e **software** rappresentati come cerchi concentrici.
- L'hardware è rappresentato dal cerchio più interno e il software applicativo da quello più esterno.
- In applicazioni complesse ci sono spesso **più livelli di applicazioni software**.
- Per esempio, un **server Web (Apache)** che usa un **database (mySQL)**. Entrambi sono eseguiti al di sopra del **software di sistema (e.g., Linux)**

Compilatore & Assemblatore

- Il **programma** (ad esempio in C) è compilato in **linguaggio assembly**
- In questo corso usiamo **RISC-V**
- Il programma in RISC-V è **quindi tradotto (assemblato)** in **linguaggio macchina**.
- Inizialmente, useremo il **simulatore RARS**
- Alla fine del corso useremo anche **hardware emulato**, SO (Linux) e un compilatore (gcc)

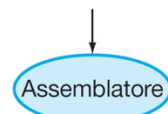
Programma
in linguaggio
ad alto livello
(in C)

```
scambia(size_t v[], size_t k)
{
    size_t temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```



Programma
in linguaggio
assembler
(per RISC-V)

```
scambia:
    slli x6, x11, 2
    add x6, x10, x6
    lw x5, 0(x6)
    lw x7, 4(x6)
    sw x7, 0(x6)
    sw x5, 4(x6)
    jalr x0, 0(x1)
```



Programma
in linguaggio
macchina binario
(per RISC-V)

```
00000000001101011001001100010011
000000000011001010000001100110011
000000000000000110011001010000011
000000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100011
0000000000000000100000001100111
```

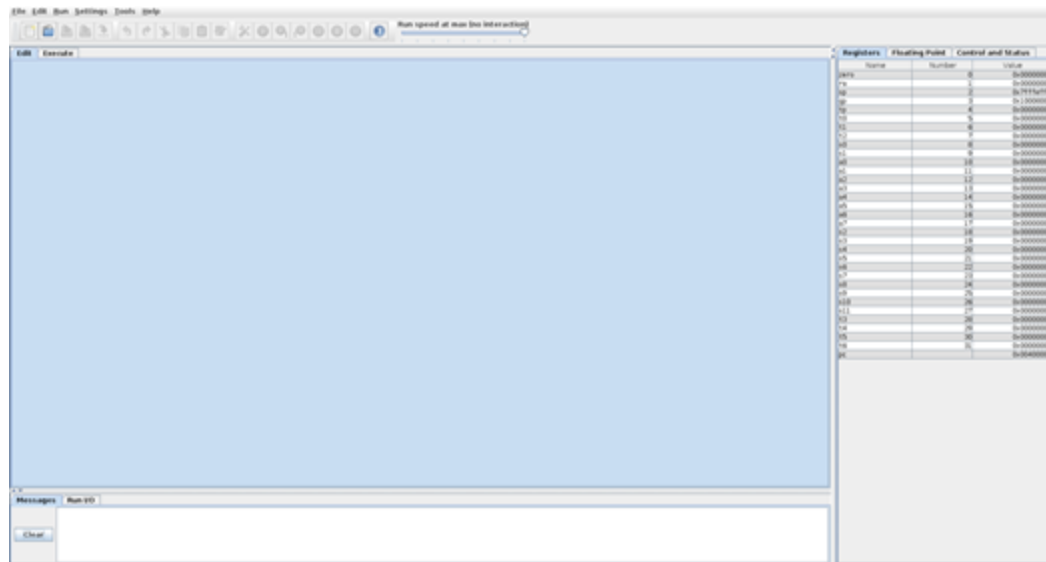
Simulatore RARS

<https://github.com/TheThirdOne/rars/wiki>

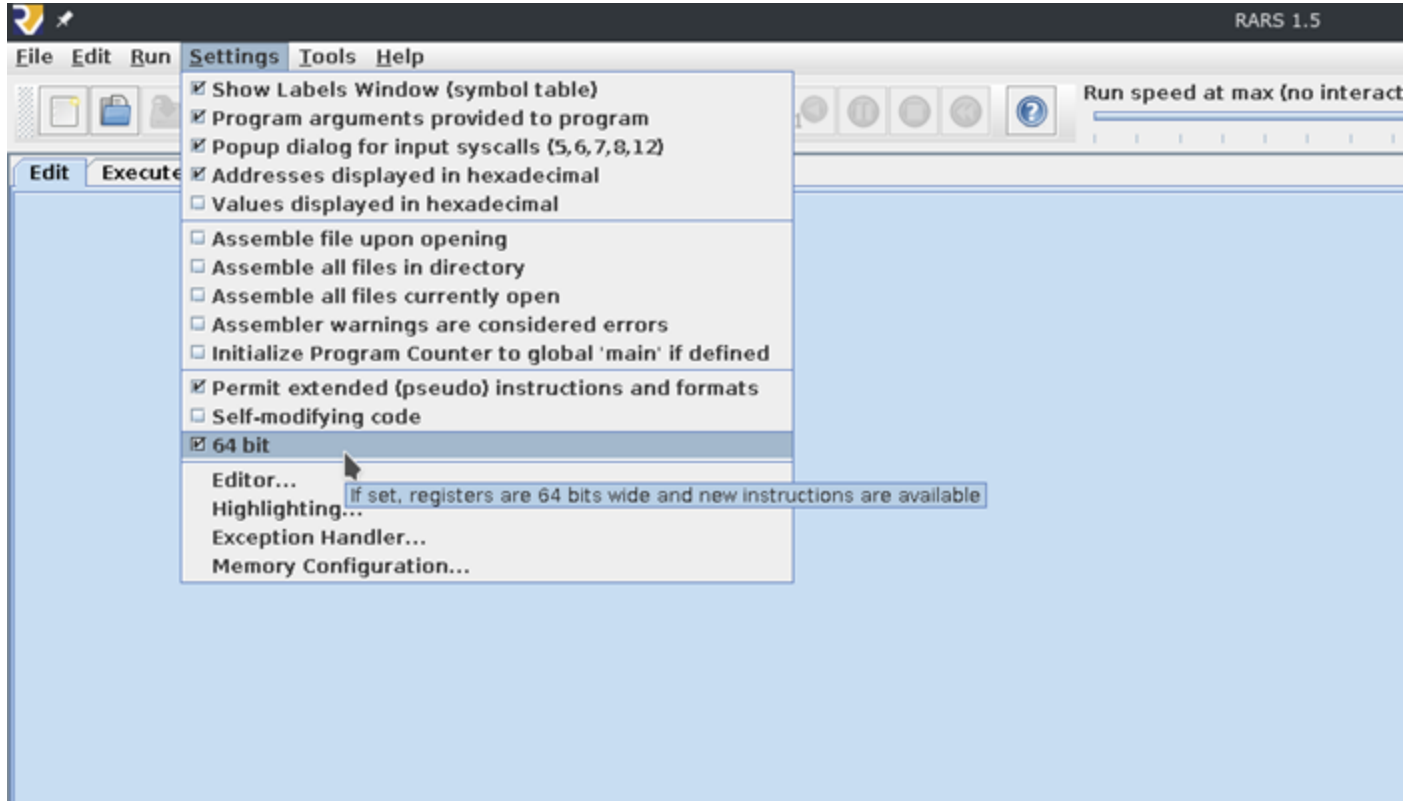
- Download:
<https://github.com/idrago/rars> **(usare la nostra "fork" – vi aiuterà per l'esame!)**
- RISC-V Assembler, Runtime and Simulator
- Download JAR
- Necessario JVM 8 per eseguirlo

Funzionalità principali

- RISC-V IMFDN Base (**riscv32 e riscv64**)
- Supporta **debugging** e **breakpoints**
- **pseudo-instruction** vs **instructions** vs **machine code**
- Statistiche della esecuzione dei programmi, e.g.,
 - a. numero di istruzioni eseguite
 - b. accessi alla memoria



RISCV32 vs RISCV64



File Edit Run Settings Tools Help

Run speed at max (no interaction)

Controlli

Edit Execute

0_template.asm

```
1 .globl _start
2
3 .text
4
5 _start:
6
7     #<assembler code here>
8
9
10 exit:
11     addi x17, x0, 10 # call number 10 = exit
12     ecall
13
```

Editor

Line: 13 Column: 1 ☒ Show Line Numbers

Messages Run I/O

Clear

I/O

Name	Number	Value
zero	0	0
ra	1	0
sp	2	2147479548
gp	3	268468224
tp	4	0
t0	5	0
t1	6	0
t2	7	0
s0	8	0
s1	9	0
a0	10	0
a1	11	0
a2	12	0
a3	13	0
a4	14	0
a5	15	0
a6	16	0
a7	17	0
s2	18	0
s3	19	0
s4	20	0
s5	21	0
s6	22	0
s7	23	0
s8	24	0
s9	25	0
s10	26	0
s11	27	0
t3	28	0
t4	29	0
t5	30	0
t6	31	0
pc		4194304

Registri

Registri

ABI Name
(ABI = Application Binary Interface)

"Proper names" (e.g., x1, ... x31)

Registro	Nome	Utilizzo
x0	zero	La costante 0
x1	ra	Indirizzo di ritorno
x2	sp	Puntatore a stack
x3	gp	Puntatore globale
x4	tp	Puntatore a thread
x5-x7	t0-t2	Temporanei
x8	s0/_fp	Salvato/puntatore a frame
x9	s1	Salvato
x10-x11	a0-a1	Argomenti di funzione/valori restituiti
x12-x17	a2-a7	Argomenti di funzione
x18-x27	s2-s11	Registri salvati
x28-x31	t3-t6	Temporanei

Name	Number	Value
zero	0	0
ra	1	0
sp	2	2147479548
gp	3	268468224
tp	4	0
t0	5	0
t1	6	0
t2	7	0
s0	8	0
s1	9	0
a0	10	0
a1	11	0
a2	12	0
a3	13	0
a4	14	0
a5	15	0
a6	16	0
a7	17	0
s2	18	0
s3	19	0
s4	20	0
s5	21	0
s6	22	0
s7	23	0
s8	24	0
s9	25	0
s10	26	0
s11	27	0
t3	28	0
t4	29	0
t5	30	0
t6	31	0
pc		4194304

Allocazione della memoria per programmi e dati nel RISC-V

- Sarà visto nel dettaglio durante la teoria
- Gli **indirizzi** indicati sono frutto solo di convenzioni software
- Lo **stack pointer** decresce verso il basso verso il segmento dati
- Il **segmento di testo** contiene il **codice del programma**
- I **dati statici e dinamici** sono "sopra" (ad indirizzi maggiori) il segmento testo



File Edit Run Settings Tools Help

Run speed at max (no interaction)

Edit Execute

Text Segment

Program Arguments:

Bkpt	Address	Code	Basic	Source
	0x00400000	0x00a00693	addi x17,x0,10	11: addi x17, x0, 10 # call numbe...
	0x00400004	0x00000073	ecall	12: ecall

Labels

Label	Address
(global)	
start	0x00400000
0_template.asm	
exit	0x00400000

☒ Data ☒ Text

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0	0	0	0	0	0	0	0
0x10010020	0	0	0	0	0	0	0	0
0x10010040	0	0	0	0	0	0	0	0
0x10010060	0	0	0	0	0	0	0	0
0x10010080	0	0	0	0	0	0	0	0
0x100100a0	0	0	0	0	0	0	0	0
0x100100c0	0	0	0	0	0	0	0	0
0x100100e0	0	0	0	0	0	0	0	0
0x10010100	0	0	0	0	0	0	0	0
0x10010120	0	0	0	0	0	0	0	0
0x10010140	0	0	0	0	0	0	0	0

0x10010000 (.data) ☒ Hexadecimal Addresses ☐ Hexadecimal Values ☐ ASCII

Registers Floating Point Control and Status

Name	Number	Value
zero	0	0
ra	1	0
sp	2	2147479548
gp	3	268468224
tp	4	0
t0	5	0
t1	6	0
t2	7	0
s0	8	0
s1	9	0
a0	10	0
a1	11	0
a2	12	0
a3	13	0
a4	14	0
a5	15	0
a6	16	0
a7	17	0
s2	18	0
s3	19	0
s4	20	0
s5	21	0
s6	22	0
s7	23	0
s8	24	0
s9	25	0
s10	26	0
s11	27	0
t3	28	0
t4	29	0
t5	30	0
t6	31	0
pc		4194304

Messages Run I/O

Assemble: assembling /home/idrago/academic/teaching/2022/arch_i/labs/lab2/0_template.asm

Assemble: operation completed successfully.

Clear

Segmento Testo (codice tradotto)

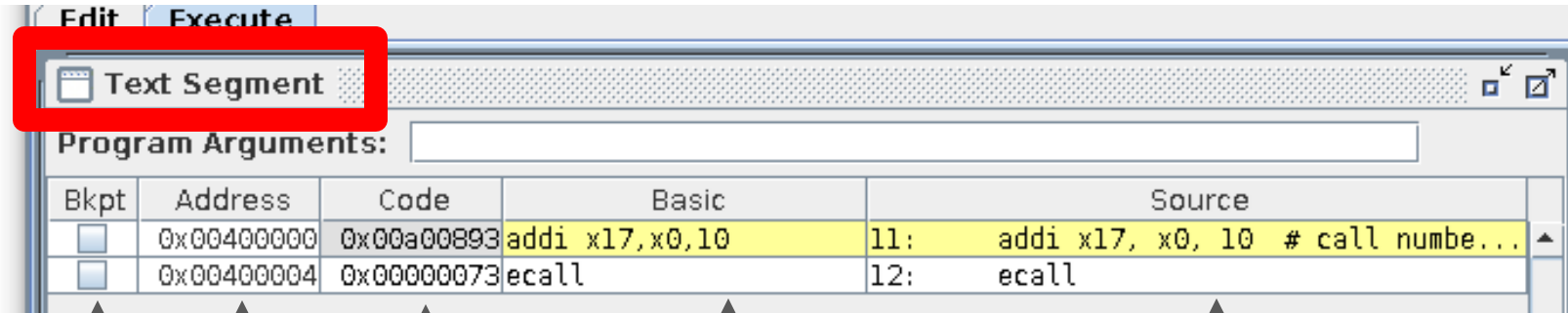
Etichette

Segmento Dati

Registri

I/O

Segmento Testo



breakpoint

indirizzi delle
istruzioni
(e.g., per il PC)

codifica dell'istruzione
(linguaggio di macchina)

RISC-V "basico",
tradotto durante assemblaggio

sorgente, include per esempio le
"pseudoistruzioni"

Segmento Dati

Visualizzazione della memoria, non solo dei "dati statici"

The screenshot shows a debugger's 'Data Segment' window. The window displays a table of memory addresses and their values. The address range shown is from 0x10010000 to 0x10010140. All values are 0. Below the table, there are navigation buttons (left and right arrows) and a dropdown menu showing the current selection '0x10010000 (.data)'. To the right of the dropdown are checkboxes for 'Hexadecimal Addresses' (checked), 'Hexadecimal Values' (unchecked), and 'ASCII' (unchecked). Below the Data Segment window, there is a 'Messages' window with a 'Run I/O' button and a 'Clear' button. A dropdown menu is open, showing a list of memory addresses and their segments: 0x10000000 (.extern), 0x10010000 (.data), 0x10040000 (heap), current gp, current sp, 0x00400000 (.text), and 0xffff0000 (MMIO).

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	V
0x10010000	0	0	0	0	0	0	
0x10010020	0	0	0	0	0	0	
0x10010040	0	0	0	0	0	0	
0x10010060	0	0	0	0	0	0	
0x10010080	0	0	0	0	0	0	
0x100100a0	0	0	0	0	0	0	
0x100100c0	0	0	0	0	0	0	
0x100100e0	0	0	0	0	0	0	
0x10010100	0	0	0	0	0	0	
0x10010120	0	0	0	0	0	0	
0x10010140	0	0	0	0	0	0	

Stack

↓

↑

Dati dinamici

Dati statici

Testo

Riservato

0x10010000 (.data)

0x10000000 (.extern)

0x10010000 (.data)

0x10040000 (heap)

current gp

current sp

0x00400000 (.text)

0xffff0000 (MMIO)

Hexadecimal Addresses

Hexadecimal Values

ASCII

Messages

Run I/O

Clear

Assembly: assembly

Assembly: operation

c:/teaching/2022/arch_i/labs/lab2/0_template.asm

Edit Execute

Text Segment

Program Arguments:

Bkp	Address	Code	Basic	Source
☐	0x00400000	00a00893	addi x17,x0,10	11: addi x17, x0, 10 # call numbe...
☐	0x00400004	00000073	ecall	12: ecall

Labels

Label	Address
(global)	
start	0x00400000
0_template.asm	
exit	0x00400000

☒ Data ☒ Text

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00400000	0x00a00893	0x00000073	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00400020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00400060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00400080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x004000a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x004000c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x004000e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00400100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00400120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00400140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

0x00400000 (.text) Hexadecimal Address ☒ Hexadecimal Values ASCII

RISC-V - Le basi

- Commenti
- Etichette (Labels)
- Direttive assembler (Assembler Directives)
- Istruzioni
 - Pseudo istruzioni (extended)
 - Operandi
 - Registri
 - Dati statici
- Environment Calls (ecall)

- Tutorial
<https://github.com/TheThirdOne/rars/wiki>
- Direttive assembler
<https://github.com/TheThirdOne/rars/wiki/Assembler-Directives>
- Istruzioni
<https://github.com/TheThirdOne/rars/wiki/Supported-Instructions>
- Environment calls (viste dopo)
<https://github.com/TheThirdOne/rars/wiki/Environment-Calls>

RISC-V - Commenti

```
# this is a comment
.globl _start

    addi x1, x0, 1    # this is another comment
```

- "#" marca l'inizio dei commenti
- Valido sia all'inizio della riga di codice sia dopo un'istruzione

RISC-V - Etichette

```
mylabel:
```

```
    # Instructions here are reached using the 'mylabel' label
```

```
mydata:
```

```
    # Data declared here can be referenced using the 'mydata' label
```

- Un'etichetta deve essere scritta come:

```
nome:
```

- Le **etichette** identificano offsets (scostamenti) nella memoria, che ci permettono di recuperare l'indirizzo al quale reperire **codici o dati**
- **Sono usate per le istruzioni di salto, nelle procedure, funzioni ecc.**
- Alcune etichette sono particolari:
 - **_start:** marca l'indirizzo della prima istruzione del nostro programma (punto di ingresso usato da Linux, il nostro "main")

RISC-V – Direttive assembler

Una componente importante dei programmi assembly sono le "**direttive**" che indicano come l'assemblatore interpreterà il codice. Le direttive **iniziano con un punto (.)** seguito da una parola chiave.



Direttiva	Funzione
.globl <label>	Dichiara un'etichetta come globale
.text	Inizio del programma, i.e., zona nella memoria dove ci sarà il codice assembly
.data	Zona nella memoria dove ci saranno i dati
.word <data>	Indica che il valore in "data" deve essere rappresentato con 32-bit (word)
.eqv <NAME> <VALUE>	Definisce una costante , ad esempio, .eqv NUMBER 4

RISC-V

Le istruzioni

Linguaggio assembler RISC-V

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Aritmetiche	Somma	add x5, x6, x7	$x5 = x6 + x7$	Operandi in tre registri
	Sottrazione	sub x5, x6, x7	$x5 = x6 - x7$	Operandi in tre registri
	Somma immediata	addi x5, x6, 20	$x5 = x6 + 20$	Utilizzata per sommare delle costanti
Trasferimento dati	Lettura parola	lw x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di una parola da memoria a registro
	Lettura parola, senza segno	lwu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una parola senza segno da memoria a registro
	Memorizzazione parola	sw x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di una parola da registro a memoria
	Lettura mezza parola	lh x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una mezza parola da memoria a registro
	Lettura mezza parola, senza segno	lhu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una mezza parola senza segno da memoria a registro
	Memorizzazione mezza parola	sh x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di una mezza parola da registro a memoria
	Lettura byte	lb x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di un byte da memoria a registro
	Lettura byte, senza segno	lbu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di un byte senza segno da memoria a registro
	Memorizzazione byte	sb x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di un byte da registro a memoria
	Lettura di una parola e blocco	lr.d x5, (x6)	$x5 = \text{Memoria}[x6]$	Caricamento di una parola come prima fase di un'operazione atomica sulla memoria
	Memorizzazione condizionata di una parola	sc.d x7, x5, (x6)	$\text{Memoria}[x6] = x5; x7 = 0/1$	Memorizzazione di una parola come seconda fase di un'operazione atomica sulla memoria
	Caricamento costante nella mezza parola superiore	lui x5, 0x12345	$x5 = 0x12345000$	Caricamento di una costante su 20 bit nei 12 bit più significativi di una parola
Logiche	And	and x5, x6, x7	$x5 = x6 \& x7$	Operandi in tre registri; AND bit a bit
	Or inclusivo	or x5, x6, x8	$x5 = x6 x8$	Operandi in tre registri; OR bit a bit
	Or esclusivo (Xor)	xor x5, x6, x9	$x5 = x6 \wedge x9$	Operandi in tre registri; XOR bit a bit
	And immediato	andi x5, x6, 20	$x5 = x6 \& 20$	AND bit a bit tra un operando in registro e una costante

Figura 2.1 Il linguaggio assembler del RISC-V esaminato in questo capitolo. Si possono trovare queste informazioni anche nella prima colonna della scheda tecnica riassuntiva del RISC-V in fondo al libro. *(continua)*

RISC-V

Le istruzioni

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Logiche (continua)	Or esclusivo immediato	ori x5, x6, 20	$x5 = x6 \mid 20$	OR bit a bit tra un operando in registro e una costante
	Xor immediato	xori x5, x6, 20	$x5 = x6 \wedge 20$	XOR bit a bit tra un operando in registro e una costante
Scorrimento (shift)	Scorrimento logico a sinistra	sll x5, x6, x7	$x5 = x6 \ll x7$	Scorrimento a sinistra mediante registro
	Scorrimento logico a destra	srl x5, x6, x7	$x5 = x6 \gg x7$	Scorrimento a destra mediante registro
	Scorrimento aritmetico a destra	sra x5, x6, x7	$x5 = x6 \gg x7$	Scorrimento a destra aritmetico mediante registro
	Scorrimento logico a sinistra immediato	slli x5, x6, 3	$x5 = x6 \ll 3$	Scorrimento a sinistra mediante costante
	Scorrimento logico a destra immediato	srl i x5, x6, 3	$x5 = x6 \gg 3$	Scorrimento a destra mediante costante
	Scorrimento aritmetico a destra immediato	srai x5, x6, 3	$x5 = x6 \gg 3$	Scorrimento a destra aritmetico mediante costante
Salti condizionati	Salta se uguale	beq x5, x6, 100	Se $(x5 == x6)$ vai a PC+100	Test di uguaglianza; salto relativo al PC
	Salta se non è uguale	bne x5, x6, 100	Se $(x5 != x6)$ vai a PC+100	Test di disuguaglianza; salto relativo al PC
	Salta se minore di	blt x5, x6, 100	Se $(x5 < x6)$ vai a PC+100	Comparazione di minoranza; salto relativo al PC
	Salta se maggiore o uguale di	bge x5, x6, 100	Se $(x5 \geq x6)$ vai a PC+100	Comparazione di maggioranza o uguaglianza; salto relativo al PC
	Salta se minore di, senza segno	bltu x5, x6, 100	Se $(x5 < x6)$ vai a PC+100	Comparazione di minoranza senza segno; salto relativo al PC
	Salta se maggiore o uguale di, senza segno	bgeu x5, x6, 100	Se $(x5 \geq x6)$ vai a PC+100	Comparazione di maggioranza o uguaglianza senza segno; salto relativo al PC
Salti incondizionati	Salta e collega	jal x1, 100	$x1 = PC+4$; vai a PC+100	Chiamata a procedura con indirizzamento relativo al PC
	Salta e collega mediante registro	jalr x1, 100(x5)	$x1 = PC+4$; vai a x5+100	Ritorno da procedura; chiamata indiretta

Figura 2.1 Il linguaggio assembler del RISC-V esaminato in questo capitolo. (continua)

RISC-V - I registri

Libro →

Registro	Nome	Utilizzo
x0	zero	La costante 0
x1	ra	Indirizzo di ritorno
x2	sp	Puntatore a stack
x3	gp	Puntatore globale
x4	tp	Puntatore a thread
x5-x7	t0-t2	Temporanei
x8	s0_/_fp	Salvato/puntatore a frame
x9	s1	Salvato
x10-x11	a0-a1	Argomenti di funzione/valori restituiti
x12-x17	a2-a7	Argomenti di funzione
x18-x27	s2-s11	Registri salvati
x28-x31	t3-t6	Temporanei

I nomi sono più informativi

Nostro primo programma RISC-V

```
# marking where execution starts (main)
.globl _start

.text
# assembly code comes in the '.text' section

# the label provided to .globl above is declared here
_start:

    # << your program here!! >>

# env call to terminate the program (explained later)
exit:
    addi x17, x0, 10  # call number 10 = exit
    ecall
```

#	→ commenti
.globl	→ simboli globali
.text	→ segmento testo
.data	→ segmento dati
exit:	→ etichetta
_start:	→ etichetta ("main")

Esercizio 1 - Somma numeri

- Scrivere un programma per caricare le costanti 41, 43, 47 nei registri **t1**, **t2** e **t3** e calcolare la loro somma. Il risultato va scritto nel registro **t4**.
- Eseguire il programma nel simulatore RARS. Eseguire un'istruzione alla volta e verificare i valori dei registri.

Esercizio 2 - Overflow

li - Carica immediato
R[rd] = costante

Supponiamo che i registri x5 e x6 contengano i numeri 0x80000000 e 0xD0000000.
Usare il simulatore RARS per rispondere alle domande.

- Quale sarà il contenuto di x30 dopo l'esecuzione di questa istruzione
`add x30, x5, x6`
Il contenuto di x30 è corretto, o si è verificato un overflow?
- Quale sarà il contenuto di x30 dopo l'esecuzione di questa istruzione:
`sub x30, x5, x6`
Il contenuto di x30 è corretto o si è verificato un overflow?
- Quale sarà il contenuto di x30 dopo l'esecuzione di queste due istruzioni:
`add x30, x5, x6`
`add x30, x30, x5`
Il contenuto di x30 è corretto o si è verificato un overflow?

Attenzione: Istruzione vs Pseudoistruzione

Address	Code	Basic	Source
			1: .globl start
			2: .text
			3: start:
0x00400000	0x00989237	lui x4,0x00000989	4: li x4, 10000000
0x00400004	0x68020213	addi x4, x4, 0x00000680	
0x00400008	0x02900293	addi x5, x0, 0x00000029	5: addi x5, x0, 41

- li - "Load immediate", carica una word, quindi 32bit
- Però le istruzioni "immediate" (e.g., addi) hanno un parametro da 12 bit